04-Express-API/02-get-requests.md

📥 GET Requests

🎯 Retrieving Data

GET - Read Resources

Learn route parameters and query strings

🛣️ Route Parameters

Dynamic URL Segments

Route parameters allow dynamic parts in URLs:

```
// Route with parameter
app.get('/api/courses/:id', (req, res) => {
    res.send(req.params.id);
});
```

Test it:

- `http://localhost:3000/api/courses/1` → Returns: `"1"`
- `http://localhost:3000/api/courses/42` → Returns: `"42"`

Access Parameters

```
req.params.id  // The value from :id
```

📝 Multiple Parameters

More Than One Parameter

```
app.get('/api/posts/:year/:month', (req, res) => {
    res.send(req.params);
});
```

Test: `http://localhost:3000/api/posts/2021/12`

Response:

```
{
  "year": "2021",
  "month": "12"
}
```

🔍 Query String Parameters

Optional Parameters

Query strings are used for optional data:

```
app.get('/api/posts/:year/:month', (req, res) => {
    res.send(req.query);
});
```

URL: `http://localhost:3000/api/posts/2021/12?sortBy=name&limit=10`

Response:

```
{
  "sortBy": "name",
  "limit": "10"
}
```

Accessing Query Params

```
req.query.sortBy  // "name"
req.query.limit   // "10"
```

🎯 GET One Course Example

Setup: Course Array

```
const courses = [
    { id: 1, name: 'course1' },
    { id: 2, name: 'course2' },
    { id: 3, name: 'course3' }
];
```

Endpoint: Get All Courses

```
app.get('/api/courses', (req, res) => {
    res.send(courses);
});
```

Test: `http://localhost:3000/api/courses`

Output: All three courses

🔎 Finding a Specific Course

Using Array.find()

JavaScript's `.find()` method searches arrays:

```
app.get('/api/courses/:id', (req, res) => {
    const course = courses.find(c => c.id === parseInt(req.params.id));
    res.send(course);
});
```

Important!

⚠️ `req.params.id` is a **string**, so we use `parseInt()`!

```
parseInt(req.params.id)  // Converts "1" to 1
```

⚠️ Handling Not Found (404)

Proper Error Handling

What if the course doesn't exist?

```
app.get('/api/courses/:id', (req, res) => {
    const course = courses.find(c => c.id === parseInt(req.params.id));

    // Course not found
    if (!course) {
        return res.status(404).send('Course with given ID not found');
    }

    // Course found
    res.send(course);
```

```
});
```

HTTP Status Codes

- **200** - OK (default)
- **404** - Not Found
- **400** - Bad Request
- **500** - Internal Server Error

🧪 Testing GET Endpoints

In the Browser

Test existing course:

`http://localhost:3000/api/courses/1`

Response:

```
{"id":1,"name":"course1"}
```

Test non-existing course:

`http://localhost:3000/api/courses/10`

Response:

`Course with given ID not found`

Also check the **Network tab** in browser DevTools to see the 404 status!

🔧 Complete GET Example

Full Implementation

```
const express = require('express');
const app = express();

const courses = [
    { id: 1, name: 'course1' },
    { id: 2, name: 'course2' },
    { id: 3, name: 'course3' }
];

// Get all courses
app.get('/api/courses', (req, res) => {
    res.send(courses);
});

// Get specific course
app.get('/api/courses/:id', (req, res) => {
    const course = courses.find(c => c.id === parseInt(req.params.id));

    if (!course) {
        return res.status(404).send('Course with given ID not found');
    }

    res.send(course);
});

app.listen(3000, () => {
    console.log('Listening on port 3000...');
});
```

💡 Best Practices

GET Request Guidelines

✅ **DO:** Use route parameters for required data (:id)
✅ **DO:** Use query strings for optional filters (?sortBy=name)
✅ **DO:** Return proper HTTP status codes
✅ **DO:** Handle not found cases with 404
✅ **DO:** Parse strings to numbers when needed

❌ **DON'T:** Modify data in GET requests
❌ **DON'T:** Forget to validate the ID
❌ **DON'T:** Return 200 when resource doesn't exist

🎯 Summary

GET Requests

- **Route parameters** (`:id`) for required parts of URL
- **Query strings** (`?key=value`) for optional filters
- **Array.find()** to search for items
- **parseInt()** to convert string IDs to numbers
- **404 status** when resource not found
- **Early return** to prevent further execution

🏠 Course Home | 📘 Chapter 4 Home

← Previous: REST & Express | Next: POST Requests →